

Progetto “Travel Companion”

Report del progetto – Corso di Laboratorio di Applicazioni Mobili

Anno Accademico 2024-2025

Corso di Laurea in Informatica per il Management

Università di Bologna

Docente del corso – Prof. Federico Montori

Autori

Francesco Maria Fuligni – Matricola: 0001068987

Roberto Zanolli – Matricola: 0001070505

Sommario

Overview e funzionamento	2
Architettura e interfaccia	4
Architettura e persistenza	4
User Interface	4
Dettagli di implementazione	9
Predizione dei dati futuri	9
Background Jobs e notifiche	9
Librerie e servizi	10
Permessi	11

Overview e funzionamento

L'applicazione Travel Companion consente di pianificare, registrare e analizzare i propri spostamenti direttamente da uno smartphone Android, offrendo un'interfaccia intuitiva e una serie di funzionalità integrate per monitorare con precisione ogni viaggio.

All'avvio, l'utente è invitato a creare un nuovo viaggio scegliendo tra tre diverse tipologie, differenziate in base alla durata e all'ambito:

- **In città:** destinato a brevi escursioni nel raggio della posizione attuale; la durata è impostata automaticamente fino al termine della giornata odierna e la destinazione coincide con la localizzazione corrente.
- **Un giorno:** adatto a escursioni di una singola giornata verso qualsiasi luogo desiderato; l'utente seleziona liberamente la meta su mappa o digitando l'indirizzo, mentre la data di termine viene fissata automaticamente al termine della giornata odierna.
- **Più giorni:** pensato per itinerari di durata flessibile, da due giorni fino a un periodo definito dall'utente; durante la creazione, è possibile specificare la data di fine oltre alla destinazione del viaggio (come avviene anche per viaggi di una singola giornata).

Una volta inseriti i parametri iniziali, l'app passa alla **schermata principale di registrazione del viaggio**, nella quale:

1. Viene mostrata una mappa interattiva che mostra in tempo reale la posizione dell'utente.
2. L'utente può inserire **note testuali** o **scattare fotografie**, che verranno automaticamente associate al viaggio e visualizzabili alla conclusione dello stesso.

Il viaggio si conclude secondo due modalità alternative: al superamento della data di termine inserita oppure al comando esplicito di interruzione anticipata da parte dell'utente (disponibile nella schermata di tracciamento del viaggio, a viaggio in corso). In entrambe le situazioni, l'app esegue il calcolo della distanza percorsa e salva il percorso con tutti gli elementi multimediali.

Nella sezione contenente l'**elenco dello storico dei viaggi**, ogni registrazione è elencata in ordine cronologico. Selezionando il singolo viaggio, l'utente può visualizzare tutte le immagini scattate durante l'itinerario, le note inserite, la strada percorsa e la distanza totale.

Per un'analisi più approfondita dei dati sullo storico dei viaggi, la schermata **Statistiche** offre due viste complementari:

- Una **heatmap** che evidenzia le aree maggiormente esplorate dall'utente.
- Un **grafico temporale** che riepiloga, per ognuno degli ultimi 6 mesi, i chilometri percorsi totali e il numero di viaggi intrapresi, permettendo di individuare tendenze stagionali o variazioni di comportamento nel breve periodo.

Oltre all'analisi dei dati storici, viene presentato un pannello di **predizioni** per il mese successivo: l'app stima il numero di spostamenti e i chilometri probabili, fornendo un'anteprima visiva ed una descrizione del trend previsto per il mese successivo.

Infine, l'area **Impostazioni** consente di personalizzare il proprio profilo (inserendo nome, foto e indirizzo di domicilio) e di effettuare un **reset completo** dell'app, riportandola allo stato iniziale cancellando tutti i dati salvati.

Travel Companion implementa anche un sistema di **notifiche automatiche**:

- Dopo un periodo superiore a un mese senza avviare nuovi viaggi, un promemoria incoraggia l'utente a riprendere l'attività di registrazione.
- Quando ci si avvicina a Punti di Interesse predefiniti, viene generata una notifica che invita a registrare un nuovo viaggio.
- Se è stato inserito l'indirizzo di domicilio, l'app rileva l'uscita da tale zona e suggerisce di iniziare un viaggio.

Per l'utilizzo della fotocamera esterna, il tracciamento della posizione e la ricezione delle notifiche sono richiesti i permessi all'utente, necessari per il corretto funzionamento di delle feature dell'applicazione.

Architettura e interfaccia

Architettura e persistenza

L'architettura di Travel Companion si struttura in livelli distinti per aree di responsabilità.

La **persistenza** è implementata interamente con Room: le entità Kotlin mappano le tabelle per viaggi, coordinate, foto e annotazioni, mentre le DAO espongono operazioni CRUD e query aggregate. Il database vero e proprio è inizializzato e configurato in un'unica classe, `TravelDatabase.kt`, che gestisce anche eventuali migrazioni.

Il **repository**, rappresentato da `TravelRepository.kt` si occupa di ricevere le chiamate al database, di sincronizzazioni i dati e di restituire flussi reattivi di LiveData o Flow che possono essere osservati dai livelli superiori.

I **servizi di background**, servono al tracciamento continuo del GPS, sono stati creati un Foreground Service che, con le coroutine, acquisisce coordinate a intervalli configurabili senza bloccare il thread principale né consumare eccessivamente la batteria. Le operazioni programmabili, quali il promemoria mensile o la notifica di vicinanza a punti di interesse, sono invece gestite da WorkManager.

L'**interfaccia utente** è organizzata secondo il pattern MVVM e sfrutta il Navigation Component per la navigazione fra i diversi schermi. La MainActivity funge da container e non contiene logica di presentazione: ogni funzionalità è implementata tramite un Fragment dedicato, osservabile tramite LiveData, e ViewModel. I diversi moduli – Home, NewTrip, MyTrips, Stats, TripDetails, Settings – condividono componenti comuni (quali dialog e helper per la navigazione) raccolti in un package a parte, per favorire il riuso di codice.

Il codice di supporto, ovvero le **utilities** per il calcolo delle distanze, la formattazione di date e unità e la gestione dei permessi, risiede in un package utils dedicato.

Tutte le risorse grafiche (layout XML, drawable, menu, mipmap) sono organizzate secondo le convenzioni Android.

User Interface

Container e Navigazione

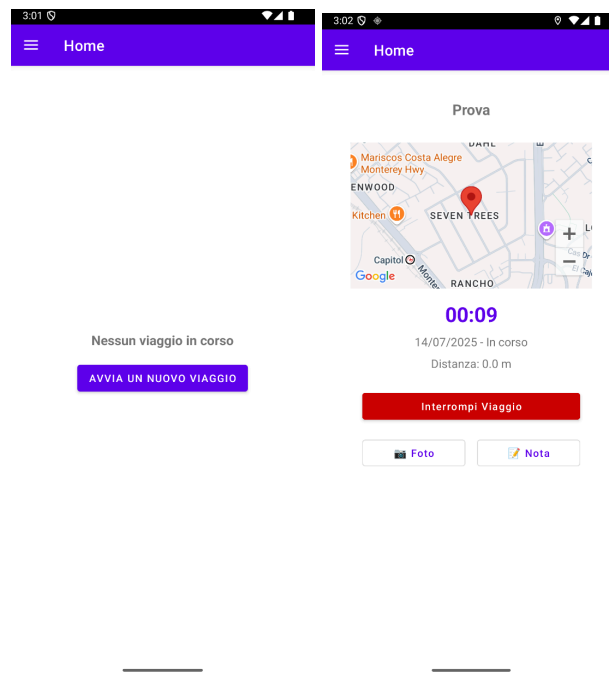
- MainActivity funge da contenitore principale.

- Una **NavigationView** (nel layout di **MainActivity**) con cinque tab (“Home”, “Nuovo viaggio”, “I miei viaggi”, “Statistiche”, “Impostazioni”) permette di eseguire il replace del fragment corrispondente, navigando tra le schermate principali

Home

Se non sono presenti viaggi in corso, mostra un fragment che permette di navigare alla creazione di un nuovo viaggio. Altrimenti, viene mostrato il fragment con i dettagli del viaggio attivo:

- Al centro del layout, una **Google Map** mostra in real-time la posizione dell’utente.
- Al di sotto di essa sono presenti tre **bottoni**, per interrompere il viaggio, aggiungere foto o inserire una nota.



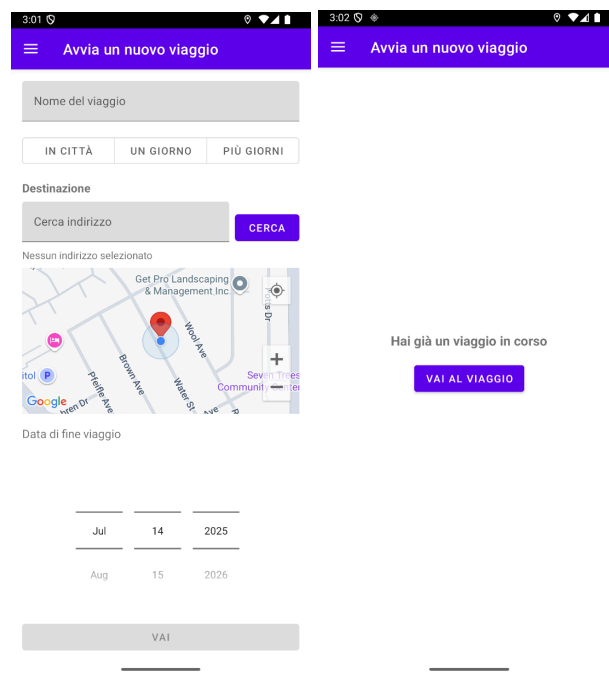
Nuovo viaggio

Mostra alternativamente un fragment per la visualizzazione della home se è già presente un viaggio in corso, altrimenti la schermata per la creazione di un nuovo viaggio contenente:

- In alto, un gruppo di **ToggleButton** presenta le tre opzioni (“In città”, “Un giorno”, “Più giorni”). La selezione dinamica abilita o disabilita i campi successivi: se si sceglie “In città”, la data di termine e la destinazione vengono settate

automaticamente; scegliendo “Un giorno”, si ha la possibilità di inserire anche una destinazione a piacere; selezionando “Più giorni”.

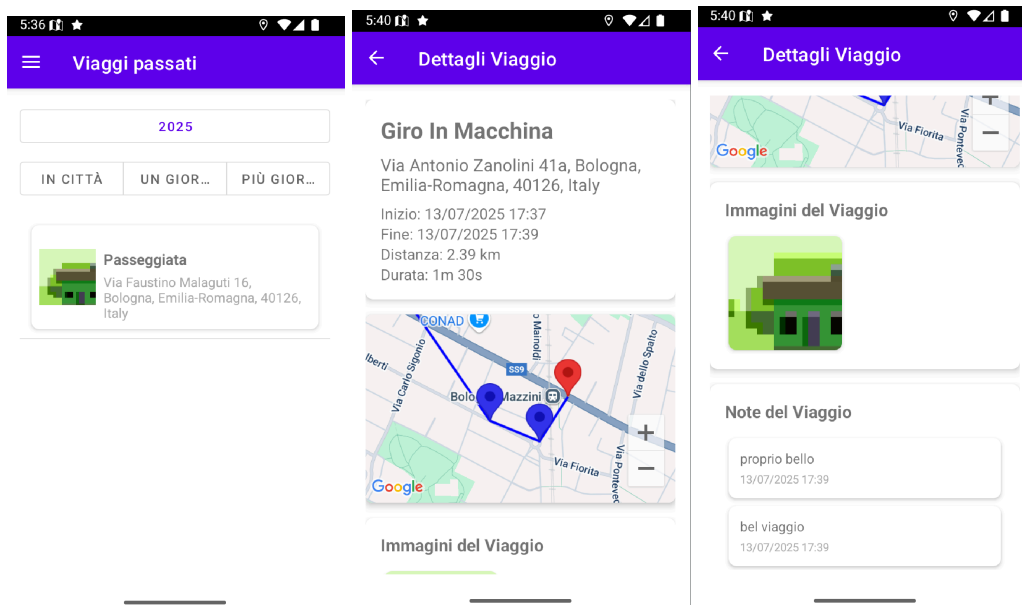
- L'utilizzo di un AddressSearchFragment, contenente una Google Map (realizzata in un apposito MapFragment) e un campo per la ricerca dell'indirizzo, permette di inserire una destinazione esistente per il viaggio (quando possibile).
- Un DatePicker, utilizzabile solamente per viaggi “Più giorni”, permette di selezionare la data di termine del viaggio.
- Una volta compilati i campi obbligatori, il bottone “Vai” diventa attivo. Al click, salva l'entità nel database tramite il repository e naviga automaticamente alla schermata Home, dove il servizio di tracciamento viene avviato.



I miei viaggi

In questa schermata l'utente consulta lo storico:

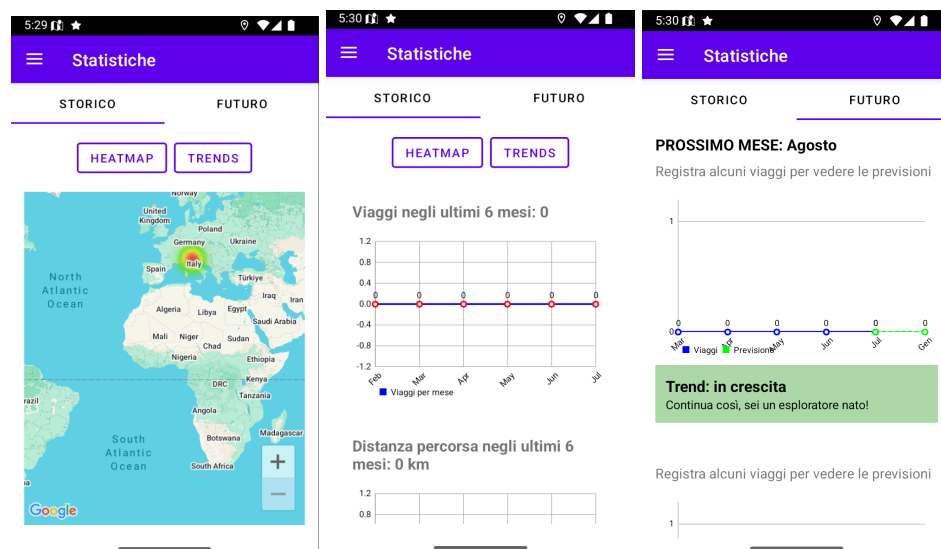
- Un RecyclerView verticale elenca ogni viaggio, usando un TripAdapter dedicato.
- L'utilizzo di uno Spinner aiuta a selezionare l'anno per cui filtrare i viaggi nella lista.
- Selezionando un elemento della lista, si visualizzano i dettagli del viaggio, quali date di inizio e fine, distanza percorsa, immagini e note salvate, percorso effettuato.



Statistiche

La sezione combina due widget, tra cui è possibile navigare grazie al TabLayout:

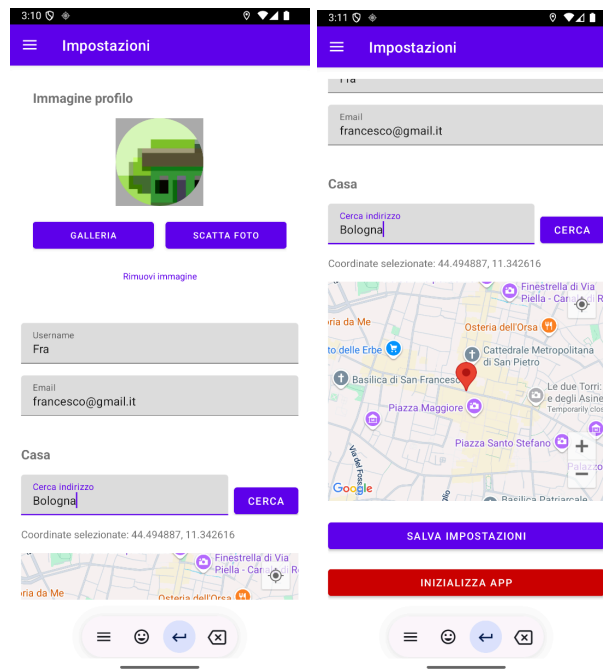
- Una Heatmap popolata con tutte le location registrate, che permette di visualizzare le aree maggiormente visitate dall'utente.
- Due Chart che visualizzano rispettivamente i chilometri e il numero di viaggi per mese.
- Una sezione apposita con grafici per previsioni sull'andamento.



Impostazioni

Qui l'utente gestisce il proprio profilo e le preferenze globali:

- Mostra immagine del profilo (con bottoni dedicati per modificarla), nome, email e indirizzo di casa.
- Un pulsante di ripristino dati invoca una funzione per cancellare il database e tutte le informazioni salvate nell'app.



Dettagli di implementazione

Predizione dei dati futuri

La logica di predizione dei dati futuri è interamente realizzata in locale. Si combinano i dati storici estratti dal database (riguardo i viaggi dei 5 mesi passati) e un modello basato sulla media mobile, per predire valori sul prossimo mese nel futuro.

- **Numero di viaggi previsti** = somma di tutti i viaggi effettuati divisa per il numero di mensilità, arrotondata all'intero più vicino.
- **Distanza prevista** = somma di tutte le distanze percorse divisa per il numero di mensilità, convertita in chilometri e arrotondata.

Per evidenziare se la tendenza è in crescita o in calo, il codice paragona il valore previsto con quello dell'ultimo mese registrato: se la previsione è inferiore all'ultimo valore storico, si considera un trend in calo (box colorato di rosso tenue), altrimenti in crescita.

In caso di assenza di dati memorizzati, si mostrano dei messaggi testuali che invitano l'utente a registrare nuovi viaggi.

Background Jobs e notifiche

L'app si avvale di due diverse tipologie di job in background: **servizi in primo piano** per il monitoraggio della posizione in tempo reale e **task schedulati** affidati al WorkManager per promemoria e notifiche periodiche, oltre all'impiego della Geofencing API per il rilevamento di uscite e ingressi dalla zona di casa, inserita dall'utente.

Il tracciamento continuo, utile per monitorare la posizione durante un viaggio, è affidato al **TrackingService**, un Foreground Service utilizza FusedLocationProvider per gli aggiornamenti della posizione. Il service aggiorna il database Room, memorizzando le location che compongono il viaggio ed incrementando distanza e durata del viaggio ad ogni iterazione. Inoltre, se la data di fine prefissata è stata superata, il servizio interrompe automaticamente il logging e termina il viaggio.

Il **LocationUpdatesService** si occupa di avvisare l'utente della presenza di luoghi di interesse nelle vicinanze. Ad intervalli definiti effettua una chiamata al Places SDK di Google ed invia una notifica push, se l'utente si trova nei pressi di un punto di interesse.

Per le notifiche periodiche, l'app utilizza il **WorkManager**, che programma un job per controllare la presenza di viaggi nell'ultimo mese: se l'utente non avvia viaggi da più di un mese, viene notificato.

HomeGeofenceService registra un'area di geofence intorno all'indirizzo di casa (quando salvato dall'utente). L'app sfrutta le API di geofencing per intercettare gli eventi di ingresso e di uscita da tale area ed inviare un promemoria all'utente per ricordare di monitorare i propri spostamenti avviando un viaggio.

NotificationUtils fornisce un meccanismo comune per strutturare le notifiche da inviare all'utente, a seconda delle varie casistiche. NotificationUtils provvede a creare il NotificationChannel per la notifica ed a configurare l'intent per portare l'utente alla schermata principale dell'app quando tocca la notifica.

Librerie e servizi

Di seguito una panoramica delle principali librerie e servizi esterni integrati nell'applicazione:

1. **Room**
Usato come livello di persistenza locale: le entità Kotlin (viaggi, coordinate, foto e annotazioni) sono mappate su tabelle, mentre le DAO espongono query sulle entità.
2. **Google Maps SDK**
Fornisce la mappa interattiva su cui tracciare la posizione in tempo reale e visualizzare itinerari e localizzazioni.
3. **Google Places SDK**
Permette all'utente di cercare e selezionare destinazioni in fase di creazione del viaggio, sfruttando l'autocomplete e i dettagli dei luoghi (indirizzo formattato, coordinate).
4. **Android Maps Utils**
Fornisce funzioni aggiuntive sul layer mappa: clustering di marker, heatmap e generazione di snapshot per le anteprime nelle card di storico.
5. **Google Play Services Location & Geofencing**

- FusedLocationProvider per acquisire coordinate GPS con consumi ottimizzati.
- Geofencing API per definire aree di interesse rispetto a cui ricevere notifiche.

6. **MPAndroidChart**

Utilizzata nella schermata Statistiche per disegnare grafici lineari (per lo storico dei viaggi e le previsioni future).

7. **Camera Intents**

L'app sfrutta l'Intent di sistema per scattare foto e salvarle nel storage interno. Le immagini sono poi recuperabili tramite Room.

8. **Glide**

Utilizzata per il caricamento e la gestione efficiente di immagini locali, in particolare per:

- Visualizzare le foto scattate durante il viaggio, nel dettaglio del viaggio.
- Caricare le miniature di immagini nei blocchi della lista (RecyclerView) dello storico dei viaggi.

Permessi

Per garantire tutte le funzionalità sono necessari i seguenti permessi:

1. **ACCESS_FINE_LOCATION**

Necessario per ottenere la posizione in alta precisione tramite GPS, utilizzato dal foreground service di tracciamento e per Geofencing.

2. **ACCESS_COARSE_LOCATION**

Fornisce posizionamento meno preciso (sfruttando rete cellulare o Wi-Fi).

3. **ACCESS_BACKGROUND_LOCATION**

Permette al servizio di tracciare la posizione anche quando l'app non è in primo piano, indispensabile per il logging continuo durante la registrazione di un viaggio.

4. **INTERNET**

Richiesto per caricare le mappe Google e per scaricare risorse da rete.

5. **ACCESS_NETWORK_STATE**

Consente di verificare la disponibilità di connessione prima di tentare operazioni di rete (come il caricamento di mappe o di luoghi dal Places SDK).

6. **CAMERA**

Abilitato quando l'utente scatta foto tramite l'Intent di sistema, necessario per acquisire nuove immagini direttamente dall'app.

7. **FOREGROUND_SERVICE**

Consente di avviare un servizio in primo piano con notifica persistente, essenziale per mantenere attivo il tracciamento GPS anche a schermata spenta o con l'app in background.